Hands-on Labs

Build Real Things with LLMs

Day 1 - Session 4

KavinSchool

Part 1: Code Explainer

Paste Code → Get Plain-English Explanations

KavinSchool

Part 1: Code Explainer

Goal: Build a tool that takes a Python function and returns a plain-English explanation.

Requirements:

1. Accept a Python code snippet (from file or paste)
2. Use a system message that instructs the model to explain like a senior developer
3. Output: function purpose, parameters, return value, notable logic
4. Format the explanation in Markdown

Part 1: Code Explainer

Example input:

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Time: 20 minutes | **File:** `01_code_explainer.py`

Part 1: Hints

System message idea:

```
system = """You are a senior Python developer explaining code to a
junior colleague. For each function:

1. **Purpose**: What does this function do? (one sentence)
2. **Parameters**: List each parameter and its expected type/purpose
3. **Return value**: What does it return?
4. **Algorithm**: Step-by-step explanation of the logic
5. **Complexity**: Time and space complexity (Big O)
6. **Edge cases**: What inputs might cause issues?

Use Markdown formatting. Be clear and concise."""
```

> **Tip:** The system message does the heavy lifting. A well-crafted system message makes simple user prompts work brilliantly.

Part 2: Bug Triager

Error Logs → Severity Classification

Part 2: Bug Triager

Goal: Build a tool that classifies error logs by severity and suggests a fix.

Requirements:

1. Accept an error log or stack trace as input
2. Classify severity: `CRITICAL` / `HIGH` / `MEDIUM` / `LOW`
3. Identify the root cause
4. Suggest a fix
5. Output structured JSON

Part 2: Bug Triager

Example input:

```
Traceback (most recent call last):
  File "app.py", line 42, in process_payment
    result = gateway.charge(amount, card_token)
  File "gateway.py", line 18, in charge
    response = requests.post(url, json=payload, timeout=5)
requests.exceptions.ConnectionError: Max retries exceeded
```

Time: 25 minutes | **File:** `02_bug_triager.py`

Part 2: Expected Output

```
{
  "severity": "HIGH",
  "category": "Network/External Service",
  "root_cause": "Payment gateway is unreachable, connection
                 refused or DNS failure after max retries.",
  "affected_component": "process_payment → gateway.charge",
  "suggested_fix": [
    "Add retry logic with exponential backoff",
    "Implement circuit breaker pattern",
    "Add fallback payment processor",
    "Improve error handling to return user-friendly message"
  ],
  "immediate_action": "Check payment gateway status page and
                       network connectivity"
}
```

> **Tip:** Ask the model to respond in JSON. Set `temperature=0.0` for consistent structure.

Part 3: User Story Generator

Feature Idea → Agile User Story

KavinSchool

Part 3: User Story Generator

Goal: Build a tool that takes a brief feature idea and generates a structured agile user story.

Requirements:

1. Accept a brief feature description as input
2. Use a system message that instructs the model to act as a Product Manager
3. Output: Title, User Story format (As a... I want to... So that...), and 3-5 Acceptance Criteria
4. Format the output clearly in Markdown

Part 3: User Story Generator

Example input:

```
Add a "forgot password" link to the login page that sends
an email with a secure reset link valid for 24 hours.
```

Time: 20 minutes | **File:** `03_user_story.py`

Part 3: Hints

System message idea:

```
system = """You are an experienced Agile Product Manager.
Convert feature ideas into well-structured user stories.

For each feature idea, provide:
1. **Title**: A clear, concise title
2. **User Story**: "As a [user], I want to [action] so that [benefit]"
3. **Acceptance Criteria**: A bulleted list of 3-5 testable criteria
4. **Edge Cases**: 1-2 edge cases to consider

Use Markdown formatting. Be professional and thorough."""
```

> **Tip:** The system message does the heavy lifting. A well-crafted system message makes simple user prompts work brilliantly.

Code Walkthrough & Peer Review

Let's Look at Solutions Together

KavinSchool

Code Review Checklist

When reviewing your lab solutions, check:

Check	Question
System message	Is it specific enough? Does it set clear expectations?
Temperature	Is it appropriate for the task? (0.0 for structured, 0.7 for creative)
Error handling	What happens if the API call fails? Rate limit?
Token awareness	Are you counting tokens? Staying within budget?
Output parsing	Are you handling the response correctly? What if JSON is malformed?
Security	Is the API key in `.env`? No hardcoded secrets?

Common Patterns You Should Know

Pattern 1: Structured Output

```
system = "Always respond in valid JSON. No explanations outside the JSON."
```

Pattern 2: Role-Based Persona

```
system = "You are a security expert specializing in OWASP Top 10."
```

Common Patterns You Should Know

Pattern 3: Constrained Response

```
system = "Classify the input as exactly one of: BUG, FEATURE, QUESTION."
```

Pattern 4: Step-by-Step Reasoning

```
user = "Think step by step. First analyze the error, then suggest a fix."
```

> These patterns will be our foundation for Day 2's deep dive into Prompt Engineering.